Predicting Data-Visible Latency and Staleness of Cached Client- and Server-Rendered Web Applications

A Prospectively Validated Path Model

ISTIAK AHMED TASHRIF, Sonargaon University, Bangladesh

Whether a web page shows its data sooner with client-side rendering (CSR) or server-side rendering (SSR), and how stale that data is, depends on which cache layer answers each request, on the network, and on how much work the server does. Measurement studies report these effects at a few operating points, and practitioners cannot carry them to their own deployment. We present a path model that predicts, per user action, the time until data becomes visible in a modern stack (Next.js with TanStack Query, the Next.js data cache, Redis and PostgreSQL), and a companion model that predicts the share of page views that display stale data under time-to-live (TTL) and push invalidation. Both models are calibrated under network emulation, frozen, and evaluated prospectively: every prediction is written before its run starts, on conditions and seeds never used for fitting, and every accepted version is replicated. Across thirteen accepted fresh tests (43,776 actions) the latency model's mean error is 0.2–1.2%, including three real internet paths (HTTP/1.1, measured bandwidth, and HTTPS with HTTP/2), a second framework (SvelteKit) and a heavier page, against 10–48% for baselines that ignore cache placement. The staleness model predicts the stale-view share within 0.006–0.013 pooled absolute error, and jointly predicts latency (0.95–1.87%) as server work grows, which shows when server-side caches pay for their staleness: with 3 ms queries, TTL caches saved under 1 ms while 40–69% of views were stale; with 100 ms of database work, push invalidation recovered most of the TTL caches' saving at 2.3% stale views. We report where prediction fails: on a consumer broadband path the model's error is 5–8% because latency tails cannot be forecast from pre-run probes; under concurrency it is 2–5%; on throttled devices initial loads at low RTT miss by up to 7%. We also report measurement pitfalls that distort such studies: network emulation shifts Chromium's connection-assignment race (11% vs 75% new-connection wins), a common test-harness setting adds two round trips to every HTTPS connection, and Apple Silicon hosts make service times load-dependent.

CCS Concepts: • **Information systems** → **Web applications**; • **Networks** → **Network performance modeling**; *Network measurement*; *Network performance analysis*.

Additional Key Words and Phrases: web performance, server-side rendering, client-side rendering, caching, cache invalidation, staleness, performance modeling, prospective validation

1 INTRODUCTION

Modern web frameworks let developers choose, per page, whether data is fetched and rendered in the browser (client-side rendering, CSR) or on the server (server-side rendering, SSR), and offer several caching layers on each path: a per-user browser cache (e.g. TanStack Query), a shared framework data cache (e.g. the Next.js data cache), and an application cache in front of the database (e.g. Redis). Each layer removes a different part of the request path, and each can show stale data after a write. Which combination is fastest for a given page and network, and what it costs in freshness, is therefore not a property of the framework but of the request path.

Measurement studies compare rendering strategies under a few network profiles [15], and studies of application caches report stale-read rates for a chosen TTL and workload [10]. Their numbers describe their operating points; a practitioner with a different round-trip time, bandwidth, cache configuration or database cost cannot tell whether the conclusion transfers. Page-load analyses

Author's address: Istiak Ahmed Tashrif, istiaktashrif@gmail.com, Department of Computer Science and Engineering, Sonargaon University, 147/I, Green Road, Panthapath, Tejgaon, Dhaka, 1205, Bangladesh.

2026. 1559-1131/2026/10-ART $15.00
https://doi.org/10.1145/nnnnnnn.nnnnnnn

based on dependency graphs [13, 25] explain where load time goes, but do not model application cache layers or data freshness.

This paper takes a predictive approach. We model the critical path of each user action (initial load or in-app navigation, CSR or SSR) as a function of the cache layer that serves it, the client and internal round-trip times, the bytes on the path and the bottleneck bandwidth, and we model staleness from the age of the data each layer returns. The models are calibrated once under network emulation and then *used to predict* conditions they have not seen. We hold ourselves to a prospective protocol: model code and parameters are frozen and hashed; the prediction for every run is written before the run starts; tests use conditions and seeds never used for fitting or diagnosis; acceptance criteria are fixed in advance; and a version that passes must pass an independent replication. Every version that failed is reported with its diagnosis.

Contributions.

(1) A cache-aware path model of data-visible latency for CSR and SSR actions (Section 5) that predicts fresh conditions within 0.2–1.2% mean error on emulated paths and three real internet paths, including measured bandwidth and HTTPS with HTTP/2, and carries over to a second framework and a heavier page when recalibrated (Section 7). It is 14 to 200 times more accurate than baselines that ignore cache placement.
(2) A staleness model for stacked browser, framework and application caches under TTL and push invalidation, validated against an oracle that compares every displayed row with the database at display time, and its joint use with the latency model to locate the regime in which server-side caches pay for their staleness (Section 8).
(3) An account of where prospective prediction breaks: consumer access links, concurrency on a shared client core, and throttled devices (Section 9).
(4) Measurement findings that distort emulation-based web-performance studies, each traced to a mechanism: network emulation shifts Chromium's connection-assignment race; a common test-harness certificate setting adds two round trips per HTTPS connection; CPU quotas and Apple Silicon core scheduling make service times load-dependent (Section 10).

2 BACKGROUND AND RELATED WORK

CSR and SSR measurements. Pati and Zaki [15] compare Next.js (SSR) with React (CSR) under three network profiles and report that SSR improves loading on slow networks. Further comparisons report page-load and Core Web Vitals metrics for CSR, SSR and framework rendering modes at single operating points [8, 9, 22], and Chen [3] proposes modular rendering with adaptive hydration to reduce SSR's hydration cost. Malavolta et al. [11] measure the effect of service-worker caching on the performance and energy of progressive web apps. None of these isolates the application cache layers or predicts conditions before measuring them; we model each action's path per cache outcome and predict prospectively.

Predicting page performance. Ramakrishnan and Kaur [19] compare statistical and machine-learning models that predict page performance from page characteristics. WProf [25] extracts dependency graphs of browser page loads, and Polaris [13] and Shandian [27] use such structure to speed loads up. These works explain or predict whole-page load metrics; ours predicts the time until application data is visible, per action and per cache outcome, from the request path, and validates the prediction prospectively on fresh conditions.

HTTP versions and measurement fidelity. Wang et al. [26], Varvello et al. [24] and Wijnants et al. [28] show that the benefit of multiplexed protocols (SPDY, HTTP/2 [23]) and of stream prioritization depends on network conditions and page structure. Rajiullah et al. [18] measure web

experience at scale on operational mobile networks. Mahimahi [14] provides record-and-replay with emulated links, and Pediaditakis et al. [16] study how faithfully network experiments can be reproduced. We use emulation in the same spirit but show in Section 10 that it shifts a browser connection-level race that matters for prediction.

Cache hierarchies and TTL theory. Che et al. [2] model hierarchical web caches; Choungmo Fofack et al. [4] and Berger et al. [1] analyze networks of TTL caches; Elsayed et al. [6] optimize TTL hierarchies under network delays. A known consequence of these models is that an upstream cache filters the request stream seen downstream. We observe this filtering in a default modern stack (Section 4) and apply, rather than extend, this theory.

Freshness and invalidation. TxCache [17] provides transactional consistency for an application data cache; Quaestor [7] provides bounded-staleness query caching for database-as-a-service with invalidation; Ji et al. [10] evaluate TTL versus invalidation for an application cache in front of PostgreSQL; Mao et al. [12] argue that TTLs are ill-suited to real-time freshness. These works study a single layer or design new consistency mechanisms. We quantify staleness across *stacked* browser, framework and application caches as deployed by default, predict it, and relate it to latency.

Search. We searched OpenAlex, arXiv, Crossref and the ACM Digital Library (October 2026; queries and screening criteria in the artifact). To our knowledge, no prior work predicts per-action data-visible latency across stacked browser, framework and application caches with prospective validation, or predicts the share of stale views across such stacked caches.

3 SYSTEM, WORKLOAD AND MEASUREMENT

Application. A product catalogue backed by PostgreSQL 17 (5,000 products, 20,000 orders, 50,000 order items, 30,000 reviews, 10,000 users). An Express API serves pages of products with aggregated order counts and ratings (a pre-aggregated query; an earlier join-based query inflated order counts for 4,922 of 5,000 products through join fan-out and is not used). Redis 7.4 caches API responses per page. The front end is Next.js 16.1 (App Router, React 19).

- **CSR path** (`/csr`): a static shell; the browser fetches the API with TanStack Query (per-user cache); navigation changes the URL with `history.pushState` and needs no server round trip unless the browser cache misses.
- **SSR path** (`/ssr`): the server fetches the API through the Next.js data cache (shared across users) and returns HTML for an initial load or a React Server Components payload for a navigation.

A second front end in SvelteKit 2 (svelte-query, shallow routing, and an in-process server cache) implements the same paths for the framework-generality test, and a variant route with roughly 300 kB of additional client JavaScript and 100 rows per page (4× the payload) implements the heavier-page test. Figure 1 shows the request paths.

Testbed and networks. All services run in Docker containers. Each link (client–web, client–API, web–API, API–data) is a separate network whose delay and rate are set with `tc netem` (half the round-trip time on each end; 10 Mbit/s on client links unless stated). A Playwright 1.56 Chromium client drives sessions. Real-network tests route the client's traffic through a remote VPS acting as a layer-3 relay (DNAT and masquerade; end-to-end TCP preserved), giving a real internet round trip of roughly 250–530 ms between client and server. One further real path runs from a home broadband connection to the server. Concurrency and CPU-throttling tests pin each service to a dedicated core of a Linux x86 host.

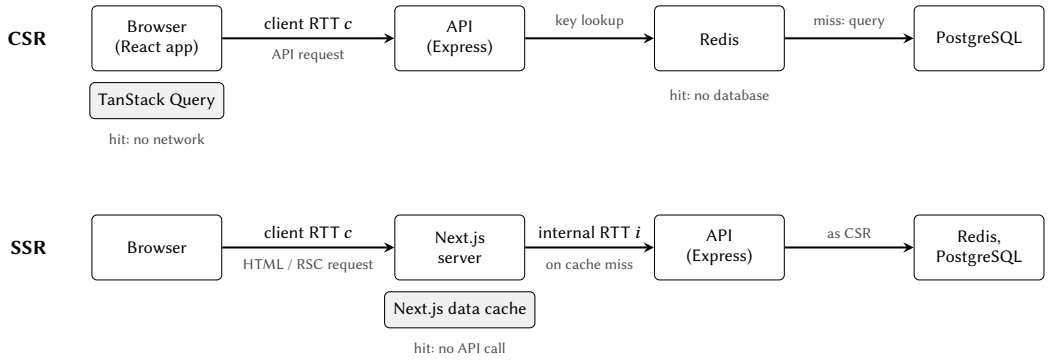


Fig. 1. Request paths of the two rendering modes through the cache layers. Each cache outcome ends the path at a different point; the model's network term counts the client round trip c and internal round trip i on the remaining path. *Initial load*: CSR fetches the document, then the JavaScript bundle (up to six connections per origin; one under HTTP/2), then calls the API on a new connection; SSR fetches the document on a new connection, and the server fetches the data before responding. *Navigation*: CSR changes the URL with `history.pushState` and makes one API request on a warm connection, or none on a TanStack hit; SSR makes one RSC request on a warm connection. Data-visible time ends at the first frame after the first data row enters the DOM.

Workloads. Each run replays a declared set of sessions: users are fresh browser contexts, each performing an initial load followed by navigations among pages drawn from a Zipf distribution (e.g. 12 users × 12 steps over 8 pages, $\alpha = 0.6$, for frequent revisits). Caches start cold in every run; a warm-up on a page no workload draws absorbs process and JIT start-up.

Endpoint: data-visible time. We measure, for each action, the time from its start (navigation start, or a performance mark immediately before an in-app navigation) until the end of the first frame after the first data row enters the DOM. A `MutationObserver` registered before any body content records the insertion; a `requestAnimationFrame` plus `MessageChannel` callback records the frame. Element Timing proved unsuitable: it reports nothing for inline text and is frame-quantized in headless Chromium.

Cache outcomes and logging. Every action is classified by the layer that served it (browser-cache hit, framework-cache hit, Redis hit, or database), using identifiers carried out of band: TanStack request ids, API logs with Redis hit/miss and database time, and framework-cache replay detection by response id. In the pilot all 10,368 actions were classified with no errors.

Writes and the staleness oracle. For the staleness experiments a seeded Poisson writer updates stock values of products on the workload's pages and logs each commit. Push invalidation deletes the Redis key and revalidates the page's framework-cache tag on each write; TTL invalidation relies on expiry (Redis TTL T, Next.js `revalidate` R with stale-while-revalidate, TanStack `staleTime` S). At the instant the first data row is inserted, the page records every displayed row's product id and stock. A view is *stale* if any displayed value differs from the database value at that instant, reconstructed from the initial snapshot and the write log.

4 PILOT OBSERVATIONS

A pilot (96 runs, 6,912 actions; 2 modes × 4 cache combinations × 2 RTTs × 2 workloads × 3 seeds) motivated the model.

- **A shared server cache leaves Redis nothing to do.** Behind the Next.js data cache, Redis's conditional hit ratio was 0.00: every repeated key is absorbed upstream. A per-user browser cache, in contrast, leaves Redis almost all of its value. This is the filtering effect predicted by cache-hierarchy theory [2, 4], here in a default configuration.
- **Initial load is decided by the rendering path, not by caches.** SSR showed data 153 ms sooner at RTT 0 and 1,093 ms sooner at RTT 170 ms, and caches changed initial load by under 1%. SSR data was visible about 756 ms before the page was interactive.
- **Navigation has a crossover.** With browser caching, CSR navigation beats SSR roughly when the browser hit rate times the client round trip exceeds the server-side work the shared cache saves.

These effects depend on RTT, cache outcome and server cost, which is what the model captures.

5 LATENCY MODEL

Structure. For action a in mode m with cache outcome o, client round trip c and internal round trip i:

$$\hat{T}(m, a, o; c, i) = T_0(m, a, o) + N(m, a, o; c, i) - N(m, a, o; 0, 0) + \rho_{m,a,o}(c), \tag{1}$$

where T_0 is the mean latency measured at RTT 0 for that path and outcome (CPU, rendering and server time), N is the critical-path network time, and ρ is an empirical residual interpolated over the client RTTs of an emulated calibration grid.

Critical paths. CSR initial: document (new connection) → JavaScript bundle → API request (new connection to the API origin). CSR navigation: one API request on a warm connection, or nothing on a browser-cache hit. SSR initial: document (new connection), plus an internal API request on a framework-cache miss. SSR navigation: one RSC request on a warm connection, plus an internal request on a miss.

Transfers. A request on a new connection pays a handshake (one RTT for TCP; two with TLS 1.3 [21]). A response of b bytes takes TCP slow-start rounds from an initial window of ten segments [5] (1,448-byte MSS), plus serialization $8b/\text{bw}$.

The JavaScript bundle and the connection race. Chromium loads the bundle's chunks over up to six connections per origin; the largest chunk runs either on the reused document connection (whose window has grown) or on a new connection (one more slow-start round). Both become free about one RTT after the request, so which one wins depends on path micro-timing. The bundle term is

$$E[\text{bundle}](c) = p\, B_{\text{new}}(c) + (1 - p)\, B_{\text{reused}}(c), \tag{2}$$

where B_{new} and B_{reused} are bundle times observed per race group in the calibration loads, and p is measured on the target path before each run from eight probe loads of a page no workload uses. Under HTTP/2 [23] all chunks share one connection, the race disappears, and the bundle term is a single calibrated curve.

Cache outcomes. Outcomes are computed analytically from the declared sessions (sequential users, cold caches, no expiry within a run), so each action's outcome is known before the run; in every test the analytic outcomes matched the logged ones (criterion C4).

Inputs per run. Frozen calibration; the declared workload; client RTT from TCP-connect probes before the run; the race probability p; and, where bandwidth is not fixed, the bottleneck bandwidth estimated from the difference of two download times (8 and 32 MB). With bandwidth as an input, the model holds two calibrated bandwidth levels and is linear in $1/\text{bw}$ between them.

Concurrency. For N concurrent users we extend the model with exact mean value analysis [20] over three single-core stations (client, web+API+Redis, database). Queue lengths come from each station's *total* CPU demand per action, measured with cgroup accounting, while each action's latency grows only by its *on-path* demand times the queue length it meets: $\hat{T}_a(N) = \hat{T}_a(1) + \sum_k P_k(a)\, s_k(N)\, Q_k(N-1)$, where $s_k(N)$ captures load-dependent demand.

6 VALIDATION PROTOCOL

Prospective design. Each model version is calibrated on emulated data, frozen (code and parameters hashed), and tested on conditions and seeds never used for fitting or diagnosis. The harness writes the prediction for each run before the run starts. A version that passes must pass an independent replication (new conditions and seeds) before it is accepted. There is no cap on versions, so failures are expected; each is diagnosed from its data, and the diagnosed data may be used to calibrate the next version, which then faces fresh tests.

Latency criteria (fixed before scoring). C1: mean relative error over cells $\leq$ 3%. C2: every cell within max(5%, 10 ms). C3: every SSR−CSR difference has the correct sign and lies within max(10 ms, 10%). C4: analytic cache outcomes match 100%. C5: every prediction written before its run started. A cell is a combination of mode, cache setting, action type and network condition; its observed value is the mean over runs (seeds).

Staleness criteria. W1: pooled mean absolute error over policy cells ≤ 0.03. W2: every cell within 0.05. W3: every pair of cells whose predicted stale shares differ by at least 0.05 is ordered correctly. W4: oracle integrity (no displayed value ahead of the database; writes logged in every run; push-purge lag p99 $\leq$ 20 ms). W5: predictions written before runs. The joint latency–staleness experiment adds L1 (mean latency error $\leq$ 3%), L2 (every cell within max(5%, 10 ms)) and L3 (each server cache's latency saving has the observed sign and lies within max(10 ms, 10%)).

7 LATENCY RESULTS

Table 1 lists every fresh prospective latency test; Fig. 2 shows all of their cells.

Emulated paths. The accepted emulated model predicts 9,216 actions per test at client RTTs of 90–320 ms and internal RTTs of 20–100 ms with 1.10% (test) and 0.77% (replication) mean error. Earlier versions failed in diagnosable ways: a state-dependent frame wait at low RTT, and round-trip costs growing slightly faster than the probed RTT, both absorbed by the residual term ρ.

Real internet paths. A model recalibrated on the server host under emulation predicted a real relayed path within 2.3%, but under-predicted CSR initial loads by 6–8%. Resource timings showed that only the bundle phase differed: on the real path the largest chunk ran on a new connection in 75% of loads, under emulation in 11% (Section 10). With the race probability measured before each run, the model predicts the real path within 0.61% and 1.08%. With the 10 Mbit/s cap removed and bandwidth probed per run, it predicts 0.62% and 0.70% (after a fix to the SSR navigation transfer term found in the first bandwidth version's replication). With HTTPS and HTTP/2 through a TLS-terminating edge, 0.26% and 0.23%.

Generality. The same equations, recalibrated, predict a SvelteKit implementation of the application (0.69%, 0.33%) and a page with roughly 300 kB more client JavaScript and four times the payload (1.23%, 0.61%).

Baselines. Baselines fitted on the same calibration data miss by 10–22% on emulated paths and by 43–48% on real paths, where initial-load latency is not linear in RTT (slow start and the connection race). The model's error is 14 to 200 times smaller (Fig. 2, right).

Table 1. Every fresh prospective latency test. Mean and max are relative errors over cells; "fails" lists failed criteria; the last column is the test's own verdict (a version is accepted only if its test and replication both pass).

Experiment	Conditions	Mean %	Max %	Fails	Passed
Emulated (v5)	netem, client 90/210 ms × internal 20/75 ms	1.10	3.78	–	yes
Emulated (v5), replication	netem, client 150/320 × internal 35/100 ms	0.77	3.64	–	yes
Real path (v7)	relay + 40 ms	0.61	1.15	–	yes
Real path (v7), replication	relay + 20 ms	1.08	2.27	–	yes
Real path, bandwidth input (v8b)	relay + 10 ms, uncapped	0.62	2.23	–	yes
replication	relay + 40 ms, 15 Mbit/s	0.70	2.38	–	yes
Real path, HTTPS + HTTP/2 (v9)	relay + 35 ms	0.26	0.44	–	yes
replication	relay + 15 ms	0.23	0.61	–	yes
HTTP/2, certificate trusted (v9b)	netem 80/330 ms	1.20	6.01	C2	no
replication	netem 200/450 ms	0.18	0.53	–	yes
SvelteKit	netem 90/300 ms	0.69	2.78	–	yes
replication	netem 200/350 ms	0.33	0.90	–	yes
Heavier page	netem 90/300 ms	1.23	2.91	–	yes
replication	netem 200/350 ms	0.61	2.27	–	yes
Slow device, 4× CPU	netem 80/200 ms	2.28	7.05	C2, C4	no
replication, 2× CPU	netem 60/180 ms	2.08	7.22	C2	no
Concurrency (P3), N = 4	netem 100 ms	3.16	3.71	C1	no
replication, N = 8	netem 100 ms	2.00	3.41	–	yes
Concurrency (P4), N = 3	netem 100 ms	2.26	6.35	C2	no
replication, N = 7	netem 100 ms	4.74	9.37	C1, C2	no
Home broadband (H3)	real home path + 5 ms	8.26	26.21	C1, C2, C3	no
replication	real + 25 ms	5.28	11.39	C1, C2	no

8 STALENESS AND THE SERVER-WORK REGIME

Model. Each piece of displayed data carries the instant τ at which it was guaranteed equal to the database. With Poisson writes at rate λ_p per page, a view displayed at t is stale with probability $1 - e^{-\lambda_p (t-\tau)}$. The layers determine τ: a database read gives $\tau = t_{\text{req}}$; a Redis TTL hit gives the fill time; a push-invalidated layer gives t_{req} minus the purge lag (its entry is dropped on every write); a framework TTL cache serves its entry and refreshes in the background after R; a TanStack revisit shows the cached entry and refetches in the background after S. The timeline replays the declared sessions with display gaps calibrated without writes. For the joint model, each view's expected probability of reaching the database (by TTL age, or by survival $e^{-\lambda_p \cdot \text{age}}$ for push layers) adds d milliseconds of server work to its latency.

Validation. The first staleness test failed on an application bug: the write hook revalidated the framework cache in every Redis-push configuration, so one cell ran a different policy. After the fix, and with eight seeds per cell and an independent write stream per run, the model was accepted (Table 2, Fig. 3).

When do server caches pay? With a fast query (about 3 ms) and a local API link, server-side TTL caches saved under 1 ms of navigation latency but made 40–69% of views stale; only the browser cache cut navigation latency (170 to 90 ms at RTT 150 ms), and push invalidation removed

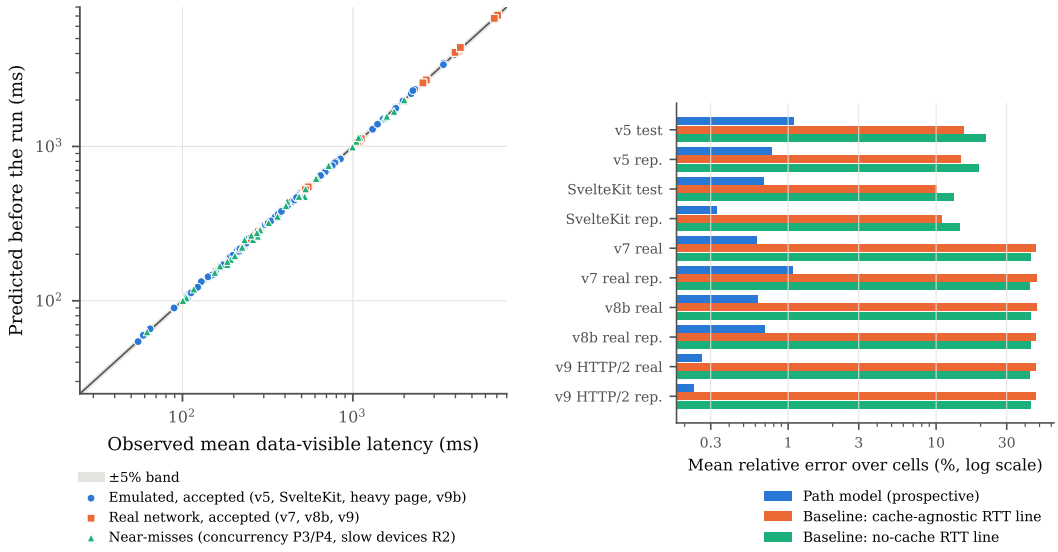


Fig. 2. Left: predicted (written before each run) vs observed mean data-visible latency for every cell of the fresh tests. Right: the model's mean error against two baselines fitted on the same calibration data: a latency–RTT line per mode and action ignoring cache layers, and the same line from cache-off runs.

Table 2. Staleness (pooled MAE, worst cell, correct orderings) and, for the regime experiment, latency (mean and worst relative error). All criteria passed in all four tests.

Test	W1 MAE	W2 max	W3	L1	L2 max
Writes, fast query: test (RTT 150, λ 3/s)	0.0113	0.0365	24/24	–	–
replication (RTT 60, λ 1.5/s)	0.0090	0.0217	22/22	–	–
Regime: test (RTT 100, d 25 ms)	0.0130	0.0383	24/24	1.87%	4.91%
replication (RTT 200, d 100 ms)	0.0062	0.0180	27/27	0.95%	2.14%

almost all staleness at no measurable cost. With 100 ms of database work per read (Fig. 3, right), SSR without caches took 328 ms per navigation at 2.8% stale views; framework and Redis push invalidation took 260 ms at 2.3%; TTL caches took about 225 ms but showed 38–58% stale views. The model predicted each of these points before the runs, so a practitioner can place a deployment between the regimes from its server cost, write rate and RTT.

9 WHERE PREDICTION FAILS

Consumer broadband. On a path from a home broadband connection to the server, three model versions missed by 5–8% mean error. The diagnosis: bandwidth varied 3–12 Mbit/s between runs; request RTT drifted by up to 22 ms within a run; and occasional slow requests (loss, retransmission, Wi-Fi episodes) raised mean navigation latency above any pre-run estimate, while typical navigations matched (e.g. median 88 ms vs 87 ms predicted). Prospective prediction is bounded here by path non-stationarity, not by the model's structure.

Concurrency. With N users sharing one client core and single-core server stations, the queueing extension predicts fresh population sizes ($N = 3, 4, 7, 8$) within 2–5% mean error (Fig. 4), and one

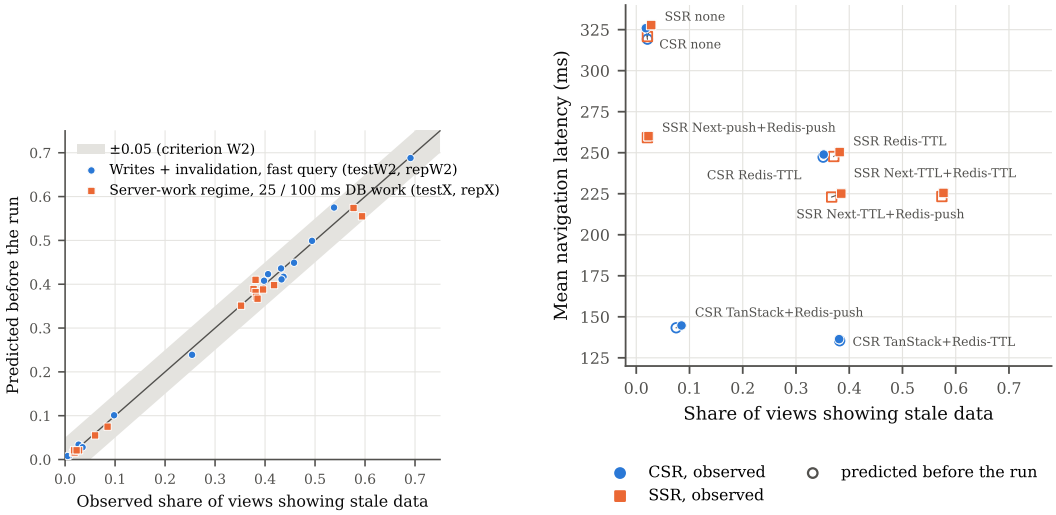


Fig. 3. Left: predicted vs observed share of stale views for every policy cell of four tests. Right: navigation latency vs stale share per caching policy with 100 ms of database work per read (RTT 200 ms); filled marks observed, hollow marks predicted before the run.

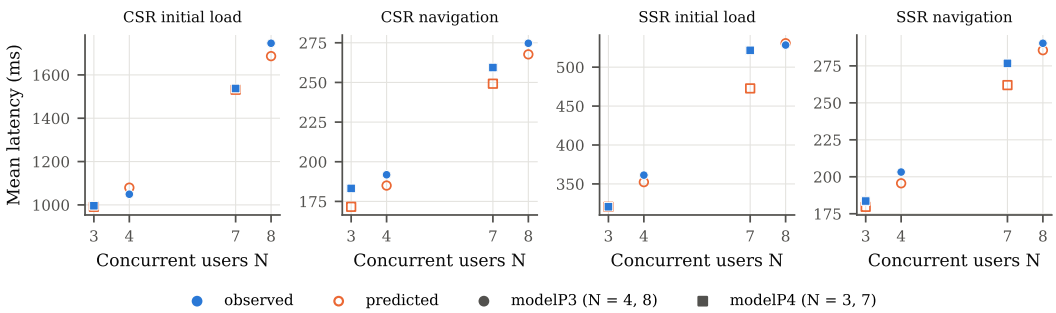


Fig. 4. Concurrency: observed and predicted mean latency per action type for fresh population sizes; two model versions (P3 at $N = 4, 8$; P4 at $N = 3, 7$).

version passed all criteria at $N = 8$, but no version passed both its test and replication. Navigations are slightly under-predicted throughout; refitting on more data moved errors between population sizes instead of reducing them.

Throttled devices. With Chromium CPU throttling calibrated at 1× and 6×, fresh tests at 4× and 2× gave 2.28% and 2.08% mean error, with all navigations within 2.3%, but initial loads at 60–80 ms RTT were over-predicted by 4–7%. These RTTs fall between calibration grid points (40 and 120 ms) where initial-load time is not linear in RTT; the HTTP/2 variant with a trusted certificate missed in the same region (Table 1). Calibration grids need points below 120 ms.

10 MEASUREMENT PITFALLS

Emulation shifts Chromium's connection race. Under `netem`, a new connection won the race for the largest chunk in 11% of loads; on the real path in 75% (per-run probes 0.50–1.00). Because the

new connection needs one more slow-start round, an emulation-calibrated model under-predicted real CSR initial loads by 6–8% until the race probability was measured on the path. Emulation-based studies of bundle loading should measure, not assume, connection assignment.

Certificate handling adds two round trips. Accepting a self-signed certificate through Playwright's `ignoreHTTPSErrors` delays every new TLS connection by about two RTTs before it starts (417 ms vs 0.8 ms at RTT 207 ms), which made CSR initial loads 29% slower (2,849 vs 2,022 ms). Trusting the certificate by key removes the delay. HTTP/2 comparisons run this way overstate HTTPS cost.

CPU quotas and Apple Silicon distort service times. CFS CPU quotas throttle bursts and produced a heavy database latency tail (p99 58 ms vs 17 ms); dedicated cores (cpusets) avoid it. On an Apple Silicon host, the same server code ran about 4.5 ms slower per database read when the machine was lightly loaded, because the hypervisor's threads moved between efficiency and performance cores; this invalidated both a concurrency experiment and a CPU-throttling experiment on that host, which we repeated on Linux. Launching many browsers at the start of short concurrency runs also contaminates measurements; a start barrier removes the effect.

11 THREATS TO VALIDITY

One application and data shape (with one heavier variant); headless Chromium only; real paths through one relay host and one home connection; analytic cache outcomes assume sequential users; models are recalibrated per framework and environment; the staleness experiments use synthetic Poisson writes to stock values. The heavier-page tests ran with 25 ms of added database work per read in both calibration and test (a leftover setting from the regime experiment), so they describe a page with that server cost.

12 CONCLUSION

Data-visible latency and staleness of cached CSR and SSR applications are predictable from the request path: which layer answers, the round-trip times, the bytes, the bandwidth and the server's work. A model calibrated under emulation and frozen predicts fresh conditions, including real internet paths and HTTP/2, within about 1%, and the staleness model locates when server-side caches pay for their freshness cost. The limits are equally clear: consumer access links, shared-core concurrency and low-RTT gaps in calibration. Prospective prediction, rather than point measurement, lets each such conclusion be checked before it is generalized.

ARTIFACT AVAILABILITY

The testbed, harness, frozen models, per-run predictions and evaluation outputs are available at https://github.com/Istiak-A-Tashrif/cached-csr-ssr-latency.

ACKNOWLEDGMENTS

The author used a generative AI tool (Claude, Anthropic) to assist with implementing the testbed and analysis code, running and analyzing the experiments, and producing the figures. The author directed the research and is responsible for all content.

REFERENCES

[1] Daniel S. Berger, Philipp Gland, Sahil Singla, and Florin Ciucu. 2014. Exact Analysis of TTL Cache Networks. *Performance Evaluation* (2014). https://doi.org/10.1016/j.peva.2014.07.001

[2] Hao Che, Ye Tung, and Zhijun Wang. 2002. Hierarchical Web Caching Systems: Modeling, Design and Experimental Results. *IEEE Journal on Selected Areas in Communications* 20, 7 (2002). https://doi.org/10.1109/JSAC.2002.801752

[3] Kaitao Chen. 2025. Improving Front-end Performance through Modular Rendering and Adaptive Hydration (MRAH) in React Applications. arXiv:2504.03884 [cs.SE]

[4] Nicaise Choungmo Fofack, Philippe Nain, Giovanni Neglia, and Don Towsley. 2014. Performance Evaluation of Hierarchical TTL-based Cache Networks. *Computer Networks* (2014). https://doi.org/10.1016/j.comnet.2014.03.006

[5] J. Chu, N. Dukkipati, Y. Cheng, and M. Mathis. 2013. Increasing TCP's Initial Window. RFC 6928. https://doi.org/10.17487/RFC6928

[6] Karim S. Elsayed, Fabien Geyer, and Amr Rizk. 2024. Utility-Driven Optimization of TTL Cache Hierarchies Under Network Delays. In *2024 IFIP Networking Conference (IFIP Networking)*. https://doi.org/10.23919/IFIPNetworking62109.2024.10619843

[7] Felix Gessert, Michael Schaarschmidt, Wolfram Wingerath, Erik Witt, Eiko Yoneki, and Norbert Ritter. 2017. Quaestor: Query Web Caching for Database-as-a-Service Providers. *Proceedings of the VLDB Endowment* 10, 12 (2017). https://doi.org/10.14778/3137765.3137773

[8] Roy Hanafi, Abd Haq, and Ninik Agustin. 2024. Comparison of Web Page Rendering Methods Based on Next.js Framework Using Page Loading Time Test. *Teknika* 13 (2024), 102–108. https://doi.org/10.34148/teknika.v13i1.769

[9] Taufan Fadhilah Iskandar, Muharman Lubis, Tien Fabrianti Kusumasari, and Arif Ridho Lubis. 2020. Comparison between Client-Side and Server-Side Rendering in the Web Development. *IOP Conference Series: Materials Science and Engineering* 801 (2020), 012136. https://doi.org/10.1088/1757-899X/801/1/012136

[10] Yunhong Ji, Xuan Zhou, Yongluan Zhou, and Ke Wang. 2023. The Case of Transparent Cache Invalidation in Web Applications. arXiv:2311.02384 [cs.DB]

[11] Ivano Malavolta, Katerina Chinnappan, Lukas Jasmontas, Sarthak Gupta, and Kaveh Ali Karam Soltany. 2020. Evaluating the Impact of Caching on the Energy Consumption and Performance of Progressive Web Apps. In *Proceedings of the IEEE/ACM 7th International Conference on Mobile Software Engineering and Systems (MOBILESoft '20)*. https://doi.org/10.1145/3387905.3388593

[12] Ziming Mao, Rishabh Iyer, Scott Shenker, and Ion Stoica. 2024. Revisiting Cache Freshness for Emerging Real-Time Applications. In *Proceedings of the 23rd ACM Workshop on Hot Topics in Networks (HotNets '24)*. ACM. https://doi.org/10.1145/3696348.3696858

[13] Ravi Netravali, Ameesh Goyal, James Mickens, and Hari Balakrishnan. 2016. Polaris: Faster Page Loads Using Fine-grained Dependency Tracking. In *13th USENIX Symposium on Networked Systems Design and Implementation (NSDI '16)*. USENIX Association.

[14] Ravi Netravali, Anirudh Sivaraman, Somak Das, Ameesh Goyal, Keith Winstein, James Mickens, and Hari Balakrishnan. 2015. Mahimahi: Accurate Record-and-Replay for HTTP. In *2015 USENIX Annual Technical Conference (USENIX ATC '15)*. USENIX Association.

[15] Swostik Pati and Yasir Zaki. 2025. Evaluating the Efficacy of Next.js: A Comparative Analysis with React.js on Performance, SEO, and Global Network Equity. In *Companion Proceedings of the ACM Web Conference 2025 (WWW Companion '25)*. ACM. https://doi.org/10.1145/3701716.3715565

[16] Dimosthenis Pediaditakis, Charalampos Rotsos, and Andrew W. Moore. 2014. Faithful Reproduction of Network Experiments. In *Proceedings of the Tenth ACM/IEEE Symposium on Architectures for Networking and Communications Systems (ANCS '14)*. 41–52. https://doi.org/10.1145/2658260.2658274

[17] Dan R. K. Ports, Austin T. Clements, Irene Zhang, Samuel Madden, and Barbara Liskov. 2010. Transactional Consistency and Automatic Management in an Application Data Cache. In *9th USENIX Symposium on Operating Systems Design and Implementation (OSDI '10)*. USENIX Association.

[18] Mohammad Rajiullah, Andra Lutu, Ali Safari Khatouni, Mah-Rukh Fida, Marco Mellia, Anna Brunstrom, Ozgu Alay, Stefan Alfredsson, and Vincenzo Mancuso. 2019. Web Experience in Mobile Networks: Lessons from Two Million Page Visits. In *The World Wide Web Conference (WWW '19)*. 1532–1543. https://doi.org/10.1145/3308558.3313606

[19] Raghu Ramakrishnan and Arvinder Kaur. 2020. An Empirical Comparison of Predictive Models for Web Page Performance. *Information and Software Technology* 123 (2020), 106307. https://doi.org/10.1016/j.infsof.2020.106307

[20] M. Reiser and S. S. Lavenberg. 1980. Mean-Value Analysis of Closed Multichain Queuing Networks. *J. ACM* 27, 2 (1980). https://doi.org/10.1145/322186.322195

[21] E. Rescorla. 2018. The Transport Layer Security (TLS) Protocol Version 1.3. RFC 8446. https://doi.org/10.17487/RFC8446

[22] Mykola Savenko and Kateryna Babii. 2025. Performance Optimization Strategies for Large-Scale Web Applications Using Next.js. *IEEE Access* 13 (2025), 217376–217386. https://doi.org/10.1109/ACCESS.2025.3647563

[23] M. Thomson and C. Benfield. 2022. HTTP/2. RFC 9113. https://doi.org/10.17487/RFC9113

[24] Matteo Varvello, Kyle Schomp, David Naylor, Jeremy Blackburn, Alessandro Finamore, and Konstantina Papagiannaki. 2016. Is the Web HTTP/2 Yet?. In *Passive and Active Measurement (PAM 2016) (Lecture Notes in Computer Science)*. Springer. https://doi.org/10.1007/978-3-319-30505-9_17

[25] Xiao Sophia Wang, Aruna Balasubramanian, Arvind Krishnamurthy, and David Wetherall. 2013. Demystifying Page Load Performance with WProf. In *10th USENIX Symposium on Networked Systems Design and Implementation (NSDI '13)*. USENIX Association.

[26] Xiao Sophia Wang, Aruna Balasubramanian, Arvind Krishnamurthy, and David Wetherall. 2014. How Speedy is SPDY?. In *11th USENIX Symposium on Networked Systems Design and Implementation (NSDI '14)*. USENIX Association.

[27] Xiao Sophia Wang, Arvind Krishnamurthy, and David Wetherall. 2016. Speeding up Web Page Loads with Shandian. In *13th USENIX Symposium on Networked Systems Design and Implementation (NSDI '16)*. USENIX Association.

[28] Maarten Wijnants, Robin Marx, Peter Quax, and Wim Lamotte. 2018. HTTP/2 Prioritization and its Impact on Web Performance. In *Proceedings of the 2018 World Wide Web Conference (WWW '18)*. 1755–1764. https://doi.org/10.1145/3178876.3186181